# Sesión 14: Animaciones e Interacciones (Animaciones Implícitas con AnimatedContainer y Explícitas con AnimationController y CurvedAnimation)

## Introducción

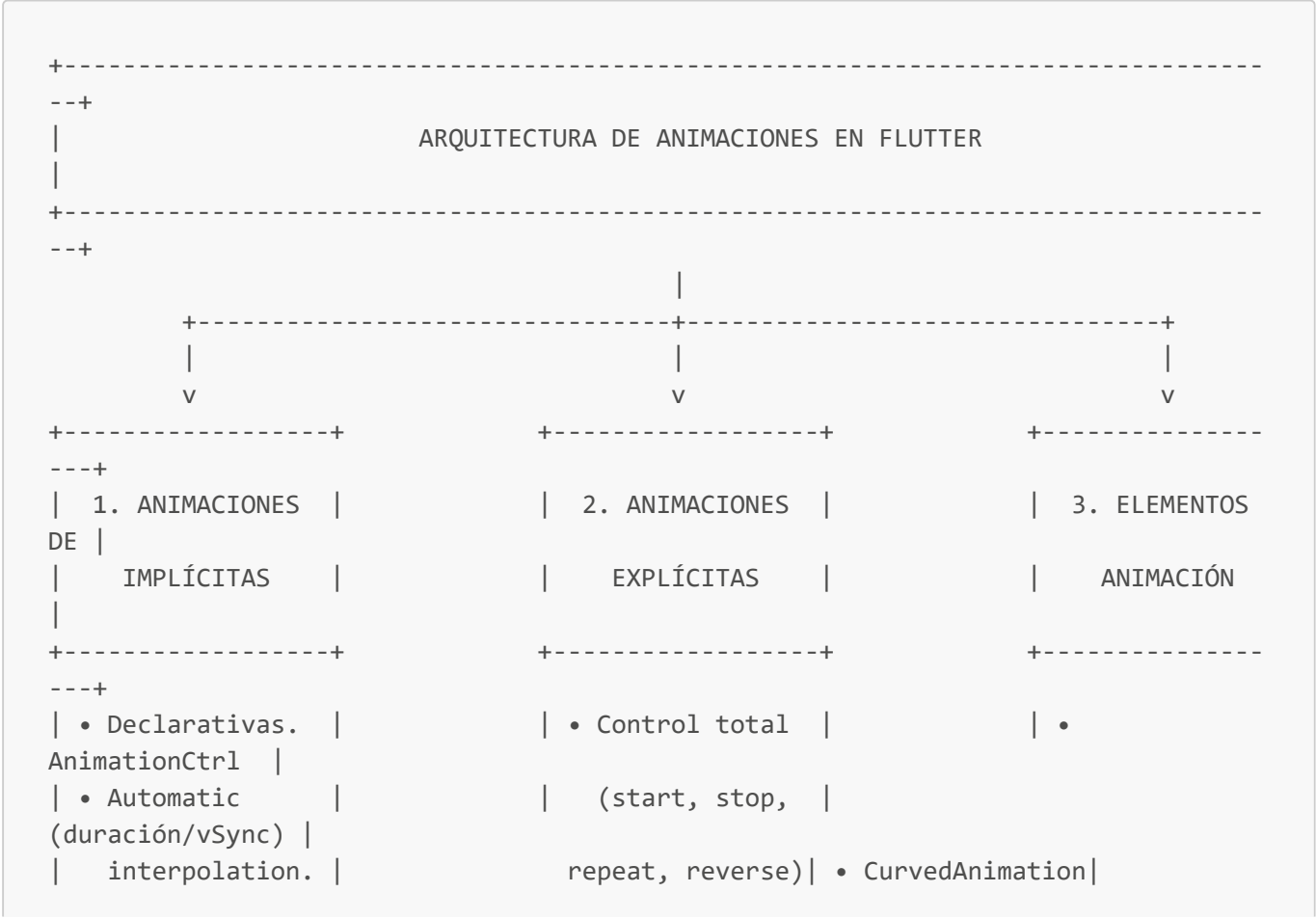
Las animaciones y transiciones dinámicas no solo aportan un valor estético a las aplicaciones móviles, sino que guían la atención visual del usuario, proporcionan retroalimentación inmediata a las interacciones y comunican cambios de estado de forma intuitiva. Flutter ofrece un motor de animación de alto rendimiento que permite desde la interpolación automática de propiedades visuales hasta la orquestación precisa de secuencias complejas.

En esta sesión abordaremos la arquitectura de animaciones e interacciones en Flutter. Analizaremos las diferencias entre el enfoque de **animaciones implícitas** (como `AnimatedContainer`, `AnimatedOpacity`, `AnimatedPadding`) y el enfoque de **animaciones explícitas** (utilizando `AnimationController`, `CurvedAnimation`, `Tween` y `SingleTickerProviderStateMixin`), proporcionando las herramientas técnicas para determinar cuándo emplear cada paradigma.

---

## Conceptos y Definiciones

El sistema de animaciones en Flutter se divide principalmente en dos modelos arquitectónicos según el nivel de control requerido sobre el ciclo de vida del movimiento.



```
| • AnimatedCntner |            | • Tickers       |
(Curves.easeIn)    |
| • duration &     |            | • Custom rebuilds| • Tween          |
|   curve.         |              vía listeners.   |              (begin/end
values) |
+------------------+            +-----------------+            +--------------
---+
```

## 1. Animaciones Implícitas (*Implicit Animations*)

- **Concepto:** Son animaciones auto-administradas y declarativas. Interpolan automáticamente la transición entre un valor antiguo y un nuevo valor cuando cambia alguna propiedad del widget mediante un re-renderizado (`setState`).
- **Widgets Comunes:** `AnimatedContainer`, `AnimatedOpacity`, `AnimatedPositioned`, `AnimatedAlign`, `AnimatedDefaultTextStyle`.
- **Propiedades Clave:**
- `duration`: Define el tiempo total que tomará la transición smooth (ej. `Duration(milliseconds: 500)`).
- `curve`: Determina la aceleración y ritmo de la interpolación mediante la clase `Curves` (ej. `Curves.easeInOut`, `Curves.bounceOut`).
- **Cuándo usarlas:** Para transiciones sencillas de cambio de estado (p. ej., expansión de tarjetas, cambio de colores, visibilidad) sin requerir iteraciones infinitas, pausas o encadenamientos complejos.

---

## 2. Animaciones Explícitas (*Explicit Animations*)

- **Concepto:** Ofrecen control total e imperativo sobre el flujo temporal y la ejecución de la animación. Permiten iniciar (`forward`), pausar (`stop`), reiniciar (`reset`), invertir (`reverse`) o repetir infinitamente (`repeat`) la animación.
- **Componentes Principales:**

1. **`AnimationController`:** Deriva de `Animation<double>` y gestiona el reloj de la animación. Genera valores continuos entre `0.0` y `1.0` durante una duración específica. Requiere un proveedor de señales de refresco de pantalla (*Ticker*).
2. **`TickerProvider` / `SingleTickerProviderStateMixin`:** Sincroniza las fluctuaciones de la animación con la tasa de refresco del panel de la pantalla (ej. 60 Hz / 120 Hz) para evitar consumo innecesario de recursos fuera de pantalla.
3. **`CurvedAnimation`:** Modifica el ritmo del flujo lineal del `AnimationController` aplicando una curva no lineal (`Curves.elasticOut`, `Curves.fastOutSlowIn`).
4. **`Tween<T>` (*In-between*):** Mapea la escala estándar `[0.0, 1.0]` producida por el controlador a un rango de tipo específico (ej. `ColorTween`, `Tween<double>(begin: 50, end: 200)` o `SizeTween`).

---

# Ejemplo Completo con Código Fuente: Tarjeta Expandible e Indicador Pulsante Animado

El siguiente código implementa una pantalla demostrativa que combina ambos enfoques: una animación implícita interactiva con `AnimatedContainer` para modificar dimensiones y bordes de una tarjeta, junto con una animación explícita controlada por `AnimationController`, `CurvedAnimation` y `Tween` para simular un radar de sincronización pulsante.

## Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoAnimacionesApp());
}

/// Widget Raíz
class DemoAnimacionesApp extends StatelessWidget {
  const DemoAnimacionesApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Animaciones e Interacciones',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaLaboratorioAnimaciones(),
    );
  }
}

/// Pantalla Principal
class PantallaLaboratorioAnimaciones extends StatefulWidget {
  const PantallaLaboratorioAnimaciones({super.key});

  @override
  State<PantallaLaboratorioAnimaciones> createState() =>
      _PantallaLaboratorioAnimacionesState();
}

class _PantallaLaboratorioAnimacionesState
    extends State<PantallaLaboratorioAnimaciones>
    with SingleTickerProviderStateMixin {
  // --- ESTADO PARA ANIMACIÓN IMPLÍCITA ---
  bool _estaExpandido = false;

  // --- COMPONENTES PARA ANIMACIÓN EXPLÍCITA ---
```

```
  late AnimationController _controladorExplícito;
  late Animation<double> _animacionEscala;
  late Animation<double> _animacionOpacidad;

  @override
  void initState() {
    super.initState();

    // 1. Inicialización del AnimationController (Duración + TickerProvider)
    _controladorExplícito = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 2),
    );

    // 2. Aplicación de la Curva de Aceleración con CurvedAnimation
    final CurvedAnimation curva = CurvedAnimation(
      parent: _controladorExplícito,
      curve: Curves.easeInOut,
    );

    // 3. Mapeo de Valores mediante Tween
    _animacionEscala = Tween<double>(begin: 0.8, end: 1.3).animate(curva);
    _animacionOpacidad = Tween<double>(begin: 0.3, end: 1.0).animate(curva);

    // Configuración para repetición continua de ciclo completo (Lazo pulsante)
    _controladorExplícito.repeat(reverse: true);
  }

  @override
  void dispose() {
    // Liberación del controlador explícito para prevenir fugas de memoria
    _controladorExplícito.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Animaciones e Interacciones'),
        backgroundColor: colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const Text(
              '1. Animación Implícita: AnimatedContainer',
              style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
            ),
```

```
            const SizedBox(height: 8),
            const Text(
              'Toca el contenedor para alternar dinámicamente sus dimensiones,
color de fondo y radio de bordes.',
              style: TextStyle(fontSize: 12, color: Colors.grey),
            ),
            const SizedBox(height: 16),

            // ANIMACIÓN IMPLÍCITA CON ANIMATEDCONTAINER
            GestureDetector(
              onTap: () {
                setState(() {
                  _estaExpandido = !_estaExpandido;
                });
              },
              child: AnimatedContainer(
                duration: const Duration(milliseconds: 600),
                curve: Curves.fastOutSlowIn,
                height: _estaExpandido ? 180 : 90,
                padding: const EdgeInsets.all(16),
                decoration: BoxDecoration(
                  color: _estaExpandido
                      ? colorScheme.primary
                      : colorScheme.surfaceContainerHigh,
                  borderRadius: BorderRadius.circular(_estaExpandido ? 24 : 8),
                  boxShadow: [
                    BoxShadow(
                      color: Colors.black.withOpacity(0.1),
                      blurRadius: _estaExpandido ? 12 : 4,
                      offset: Offset(0, _estaExpandido ? 6 : 2),
                    )
                  ],
                ),
                child: Row(
                  children: [
                    Icon(
                      _estaExpandido
                          ? Icons.unfold_less
                          : Icons.unfold_more,
                      color: _estaExpandido ? Colors.white : colorScheme.primary,
                      size: 32,
                    ),
                    const SizedBox(width: 16),
                    Expanded(
                      child: Column(
                        mainAxisAlignment: MainAxisAlignment.center,
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            _estaExpandido
                                ? 'Tarjeta Expandida (Tap para contraer)'
                                : 'Tarjeta Contraída (Tap para expandir)',
                            style: TextStyle(
                              color: _estaExpandido
```

```
                                    ? Colors.white
                                    : colorScheme.onSurface,
                                fontWeight: FontWeight.bold,
                              ),
                            ),
                            if (_estaExpandido) ...[
                              const SizedBox(height: 8),
                              const Text(
                                'Flutter auto-interpola la transición entre las
propiedades del widget de origen y destino sin controladores manuales.',
                                style: TextStyle(
                                  color: Colors.white70,
                                  fontSize: 12,
                                ),
                              ),
                            ]
                          ],
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
            const SizedBox(height: 40),

            const Text(
              '2. Animación Explícita: AnimationController & Tween',
              style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 8),
            const Text(
              'Control manual y explícito sobre la escala y opacidad con
sincronización de Ticker.',
              style: TextStyle(fontSize: 12, color: Colors.grey),
            ),
            const SizedBox(height: 24),

            // ANIMACIÓN EXPLÍCITA MEDIANTE ANIMATEDBUILDER
            Center(
              child: AnimatedBuilder(
                animation: _controladorExplícito,
                builder: (context, child) {
                  return Transform.scale(
                    scale: _animacionEscala.value,
                    child: Opacity(
                      opacity: _animacionOpacidad.value,
                      child: Container(
                        width: 120,
                        height: 120,
                        decoration: BoxDecoration(
                          shape: BoxShape.circle,
                          color: colorScheme.primary,
                          boxShadow: [
                            BoxShadow(
```

```
                            color: colorScheme.primary.withOpacity(0.4),
                            blurRadius: 20,
                            spreadRadius: 8,
                          ),
                        ],
                      ),
                      child: const Icon(
                        Icons.wifi_tethering,
                        color: Colors.white,
                        size: 50,
                      ),
                    ),
                  ),
                ),
              );
            },
          ),
        ),
        const SizedBox(height: 32),

        // CONTROLES DE MANDO PARA LA ANIMACIÓN EXPLÍCITA
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            ElevatedButton.icon(
              onPressed: () {
                if (!_controladorExplícito.isAnimating) {
                  _controladorExplícito.repeat(reverse: true);
                }
              },
              icon: const Icon(Icons.play_arrow),
              label: const Text('Iniciar'),
            ),
            OutlinedButton.icon(
              onPressed: () {
                if (_controladorExplícito.isAnimating) {
                  _controladorExplícito.stop();
                }
              },
              icon: const Icon(Icons.pause),
              label: const Text('Pausar'),
            ),
            IconButton.filledTonal(
              onPressed: () {
                _controladorExplícito.reset();
              },
              icon: const Icon(Icons.refresh),
              tooltip: 'Reiniciar',
            ),
          ],
        ),
      ],
    ),
  ),
);
```

```
  }
}
```

---

## Prácticas Guiadas / Laboratorio

### Práctica 1: Secuencia de Transición con `Hero` y `AnimatedOpacity`

1. Crea dos pantallas en la aplicación: una lista principal y una vista de detalle.
2. Envuelve una imagen o avatar con el widget `Hero` e impleméntale un parámetro identificador único (`tag: 'avatar-id'`) en ambas pantallas para generar una animación de vuelo fluida al navegar con `Navigator.push()`.
3. Integra un widget `AnimatedOpacity` dentro de la pantalla de detalle para hacer aparecer el texto descriptivo con un efecto de difuminado progresivo (*fade-in*) una vez montado el widget.

---

### Práctica 2: Implementación de Transiciones Escalonadas (*Staggered Animations*)

1. Modifica la animación explícita para coordinar múltiples propiedades que inicien en diferentes intervalos de tiempo sobre la misma línea de tiempo utilizando `Interval`.
2. Asigna la propiedad `curve` utilizando un intervalo personalizado:

```
final animacionTranslacion = Tween<Offset>(
  begin: const Offset(0, 1),
  end: Offset.zero,
).animate(
  CurvedAnimation(
    parent: _controladorExplícito,
    curve: const Interval(0.0, 0.5, curve: Curves.easeOut),
  ),
);

final animacionRotacion = Tween<double>(
  begin: 0.0,
  end: 1.0,
).animate(
  CurvedAnimation(
    parent: _controladorExplícito,
    curve: const Interval(0.5, 1.0, curve: Curves.elasticOut),
  ),
);
```

---

## Resumen y Puntos Clave

- **Animaciones Implícitas:** Son la solución más eficiente y sencilla para realizar interpolaciones de cambios de propiedad en la interfaz de usuario. No requieren gestión de estado avanzada ni

controladores manuales.

- **Animaciones Explícitas:** Otorgan un control total e imperativo sobre la ejecución, sincronización, bucles y pausa del movimiento visual.
- `AnimationController` **&** `TickerProvider`**:** Gestionan el tiempo de ejecución y garantizan el sincronismo frame por frame con la pantalla, optimizando el consumo de batería y CPU.
- `CurvedAnimation` **&** `Tween`**:** Transforman la aceleración del tiempo mediante curvas matemáticas y mapean el avance del controlador al rango de valores requeridos por los componentes.
- **Optimización de Renderizado con** `AnimatedBuilder`**:** Separa la re-evaluación del árbol de widgets del renderizado de la animación, evitando reconstrucciones globales innecesarias.

---

## Bibliografía y Referencias


1. **Google LLC.** (2024). *Introduction to animations - Flutter Docs*. Recuperado de: https://docs.flutter.dev/ui/animations
2. **Google LLC.** (2024). *Implicit animations tutorial - Flutter Cookbooks*. Recuperado de: https://docs.flutter.dev/cookbook/animation/animated-container
3. **Google LLC.** (2024). *AnimationController class documentation - Flutter API*. Recuperado de: https://api.flutter.dev/flutter/animation/AnimationController-class.html
4. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.